Centro Paula Souza
Etec Profª Ermelinda Giannini Teixeira – Santana de Parnaíba – SP
Ensino Técnico Integrado ao Médio em Desenvolvimento de Sistemas

Maria Eduarda da Silva
Natália Monteiro Takeshita
Raphael Pedro dos Santos
Thiago Alexandre da Silva Serafim
Thiago Higuti Feliputt

Manual de Desenvolvimento do projeto BuscaFarma

Santana de Parnaíba - SP
2025

Maria Eduarda da Silva
Natália Monteiro Takeshita
Raphael Pedro dos Santos
Thiago Alexandre da Silva Serafim
Thiago Higuti Feliputt

Manual de Desenvolvimento do projeto BuscaFarma

Trabalho apresentado à disciplina de Planejamento e Desenvolvimento do Trabalho de Conclusão de Curso no curso de Desenvolvimento de Sistemas
Profª Orientadora: Débora Batista da Silva de Paulo

Santana de Parnaíba - SP
2025

Sumário

Introdução.....4
Visão Geral do Sistema 4
Ambiente de Desenvolvimento 4
Ferramentas utilizadas 4
Instalação de dependências: 4
Estrutura do Repositório 5
Front End: 5
Back End:.....5
Padrões de Desenvolvimento 6
Exemplo: 6
Banco de Dados 6
Exemplo: 6
API e Regras de Negócio 7
Exemplo: 7
Instalação e Execução 7
Manutenção e Atualizações.....8
Considerações Finais 8

Santana de Parnaíba - SP

2025

Introdução

Este manual descreve os padrões, ferramentas e procedimentos utilizados para desenvolver o software BuscaFarma, incluindo ambiente, arquitetura, regras de negócio e convenções de código. Destina-se a desenvolvedores e equipes técnicas que futuramente precisarão manter ou evoluir o sistema.

Visão Geral do Sistema

O BuscaFarma é um sistema web e desktop voltado para gerenciamento e busca de medicamentos. Utiliza arquitetura cliente-servidor, API REST, banco de dados MySQL e backend em Node.js com Express e Sequelize.

Ambiente de Desenvolvimento

Ferramentas utilizadas

- Node.js 18+
- MySQL 8
- Typescript
- React
- Electron
- Git
- Visual Studio Code
- Postman

Instalação de dependências:

npm install

Estrutura do Repositório

Front End:

src/

- ├── assets/
- ├── components/
- ├── context/
- ├── hooks/
- ├── layouts/
- ├── pages/
- ├── routes/
- ├── services/
- ├── styles/
- ├── types/
- └── main.tsx

Back End:

/src/

- ├── controllers/
- ├── models/
- ├── routes/
- ├── middlewares/
- ├── interfaces/
- ├── errors/
- ├── enums/
- ├── dtos/
- ├── services/
- ├── types/
- ├── utils/
- ├── config/
- └── database/

Padrões de Desenvolvimento

- camelCase para variáveis e funções

- PascalCase para classes

- Comentários claros

- Controllers organizados por rota

Exemplo:

```
 static async logout(req: Request, res: Response) {
   try {
     const refreshToken = req.cookies.refreshToken;
     if (refreshToken) await authService.deleteRefreshToken(refreshToken);


     res.clearCookie("refreshToken");
     return respond(res, 200, null, "Usuário desconectado com sucesso");
   } catch (error) {
     logger.error("Erro ao fazer logout:", error);
     return respond(res, 500, null, "Erro ao tentar deslogar");
   }
 }
```

Banco de Dados

O banco utiliza MySQL. As principais tabelas incluem usuários, farmácias, agendamentos e estoque.

Exemplo:

```
CREATE TABLE users (

  id CHAR(36) PRIMARY KEY DEFAULT (UUID()),

  username VARCHAR(100) NOT NULL,

  email VARCHAR(100) UNIQUE,

  profileImage VARCHAR(255),
```

```
  userPassword VARCHAR(100) NOT NULL,

  phone VARCHAR(20),

  createdAt DATETIME DEFAULT CURRENT_TIMESTAMP,

  updatedAt DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

  passwordResetToken VARCHAR(255)

);
```

API e Regras de Negócio

Exemplo:

POST /entrar

Request:

```
{
  "email": "teste@gmail.com",
  "userPassword": "1234567",
  "type": "user"
}
```

Response:

```
{
    "success": true,
    "message": "Login bem-sucedido",
    "statusCode": 200,
    "data": {
        "id": "86b7f68d-78f9-4e60-a444-7a966abf8d94",
        "username": "Teste da Silva Sauro",
        "email": "teste@gmail.com",
        "accountType": "user",
        "token": …"
    }
}
```

Instalação e Execução

1. Para rodar o sistema:
2. git clone <repositório>
3. npm install
4. npm run start (backend)
5. npm run dev (frontend)

Manutenção e Atualizações

Padrões de commit:

feat: nova funcionalidade

fix: correção de bug

refactor: melhoria de código

Considerações Finais

Este manual orienta o desenvolvimento, manutenção e evolução do BuscaFarma, garantindo padronização e continuidade do projeto.